Observability in depth

PromQL rates and recording rules, scaling Prometheus, SLO burn rate alerts, logs at scale with Loki, tracing and sampling, and observability as code with cost control.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. PromQL in depth: rates, aggregation and recording rules (13 min)
2. Scaling Prometheus: HA, federation and long-term storage (13 min)
3. SLO alerting: burn rates and Alertmanager design (13 min)
4. Logs at scale: Loki, labels and retention (12 min)
5. Distributed tracing: sampling and correlation (12 min)
6. Observability as code and cost control (12 min)
Cheat sheet

Lesson 1. PromQL in depth: rates, aggregation and recording rules

13 minutes. Write correct PromQL for counters, histograms and ratios, avoid the common rate and aggregation mistakes, and use recording rules to keep dashboards fast.

PromQL looks like a simple query language, but a handful of rules decide whether your graphs tell the truth. Counters, histograms and aggregation each have a correct pattern and a common wrong one.

What you will learn

- How rate and increase handle counters and resets
- Aggregating without losing the labels you need
- Histogram quantiles and why you cannot average them
- Ratios, offsets and the staleness rules
- Recording rules for speed and consistency

Counters and rate

A counter only goes up, so its raw value is rarely useful. Use `rate(metric[5m])` to get a per-second average and `increase(metric[1h])` for a total. The range window must hold at least two scrapes, so use four times the scrape interval or more. Always apply `rate` before `sum`: summing counters first hides resets and gives wrong answers.

Aggregation and labels

`sum by (service)(rate(http_requests_total[5m]))` keeps only the service label. Use `by` to keep labels and `without` to drop a few. A ratio of two aggregations needs matching labels on both sides, or the division returns nothing. Check this when a query suddenly shows no data after a label change.

Histograms and quantiles

Histogram buckets are counters, so use `histogram_quantile(0.99, sum by (le, service)(rate(latency_bucket[5m])))`. Keep the `le` label when aggregating. Never average pre-computed quantiles from several instances: the result has no meaning. Bucket boundaries limit accuracy, so choose them around your target latency.

Staleness and gaps

A series that stops appearing is marked stale after five minutes, and a missing scrape leaves a gap. Alerts should use `for` durations and functions such as `absent()` or `up == 0` to catch missing data. A query that returns no result is not the same as zero.

Recording rules

Expensive expressions evaluated by every dashboard refresh waste CPU. A recording rule computes them on a schedule and stores the result under a new name, such as `service:http_requests:rate5m`. Alerts and dashboards then read the cheap series and stay consistent.

Diagram: PromQL in depth: rates, aggregation and recording rules

Part	What it does	What goes wrong
Application	Exposes counters.	Counter resets.
Prometheus	Scrapes and stores.	Slow queries.
Recording rules	Precompute series.	Stale results.
Dashboards and alerts	Read cheap series.	Heavy queries.

Flow: Application to Prometheus / Prometheus to Recording rules / Recording rules to Dashboards and alerts

MARGA by EknathaLabs
marga.eknathalabs.com

Incident: A dashboard that averaged quantiles

Incident: *The p99 latency graph looked fine while customers complained about slow requests.*

```
$ curl -s localhost:9090/api/v1/query --data-urlencode 'query=avg(service_latency_p99_seconds)'
{"status":"success","data":{"result":[{"value":[1731600000,"0.21"]}]}}
$ curl -s localhost:9090/api/v1/query --data-urlencode 'query=histogram_quantile(0.99, sum by (le) \
    (rate(latency_bucket[5m])))'
{"status":"success","data":{"result":[{"value":[1731600000,"1.84"]}]}}
(this is a simulated illustration, not a real incident)
```

The first query averaged the p99 of each instance, which is not the p99 of all requests. The second merges the buckets and shows a real p99 of 1.84 seconds. The dashboard was changed to use bucket aggregation and an alert was set on the correct value.

Common mistakes

- Using rate on a gauge or summing before rate
- Averaging quantiles across instances
- Dropping le from a histogram aggregation
- Using a range window shorter than two scrapes
- Putting expensive queries on every dashboard instead of recording rules

Check yourself

What should you apply before sum on a counter?

Answer: rate or increase

How do you get a correct p99 across instances?

Answer: histogram_quantile over summed buckets with the le label kept

What is a good use of a recording rule?

Answer: Precompute an expensive expression used by many dashboards and alerts

Why use a range window of at least four scrapes?

Answer: The window must contain enough samples for rate to be stable

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. Scaling Prometheus: HA, federation and long-term storage

13 minutes. Run Prometheus reliably at scale with replicas, sharding and remote write, and choose between Thanos, Mimir and federation for long retention.

A single Prometheus is easy to start and hard to keep happy as series counts grow. Scaling means choosing how to make it highly available, how to split the load and where to keep long-term data.

What you will learn

- What limits a single Prometheus: series, scrape load and memory
- High availability with replicas and deduplication
- Sharding and federation, and when each fits
- Remote write and long-term storage with Thanos or Mimir
- Capacity signals to watch

What limits one server

Memory use is driven mostly by active series, and each series costs a few kilobytes. Query load and the write ahead log also matter. A server with a few million active series needs tens of gigabytes. Watch `prometheus_tsdb_head_series` and scrape duration to see headroom before it runs out.

High availability

Run two identical replicas that scrape the same targets. Each has its own data, so a failure of one does not lose recent metrics. Alertmanager deduplicates alerts from both. Queries use a proxy such as Thanos Query that merges and deduplicates the replicas.

Sharding and federation

When one server cannot hold all series, shard targets across servers by a hash or by team. Federation lets a global server scrape selected aggregated series from others, which suits a summary view. It does not replace long-term storage, and federating raw series defeats the purpose.

Long-term storage

Thanos sidecars upload blocks to object storage and a store gateway serves them back for queries. Mimir and similar systems accept remote write and store data centrally with horizontal scaling. Both give months of retention at lower cost, and both need downsampling and compaction settings you must understand.

Capacity signals

Alert on head series growth, WAL replay time, rejected samples and slow rule evaluations. A cardinality explosion from a new label is the most common cause of sudden trouble, so keep a dashboard of the top series by metric name.

Diagram: Scaling Prometheus: HA, federation and long-term storage

Part	What it does	What goes wrong
Targets	Expose metrics.	Cardinality growth.
Prometheus replicas	Scrape and store.	Out of memory.
Query layer	Merges and deduplicates.	Slow fan out.
Object storage	Long term blocks.	Missing blocks.

Flow: Targets to Prometheus replicas / Prometheus replicas to Query layer / Prometheus replicas to Object storage / Object storage to Query layer

MARGA by EknathaLabs marga.eknathalabs.com

Incident: The label that added ten million series

Incident: *A deployment added a per-request id as a label and Prometheus ran out of memory in an hour.*

```
$ curl -s localhost:9090/api/v1/status/tsdb | head -c 220
{"status":"success","data":{"headStats":{"numSeries":11800000,"numLabelPairs":15900000},"seriesCou \
    ntByMetricName":[{"name":"http_requests_total","value":10400000}]}}
(this is a simulated illustration, not a real incident)
```

The request id label created a new series for every request. The label was dropped with a relabel rule, the server was restarted from its last good state and a limit on samples per scrape was added so one target cannot do this again.

Common mistakes

- Running one server with no replica for critical alerts
- Federating raw series instead of aggregates
- Ignoring series growth until memory runs out
- Keeping long retention on local disk only
- Adding labels with unbounded values

Check yourself

How does HA work for Prometheus?

Answer: Two replicas scrape the same targets and the query layer deduplicates

What mostly drives Prometheus memory use?

Answer: Active series

What is Thanos used for?

Answer: Long term storage and global querying over object storage

What is a common cause of sudden series growth?

Answer: A new label with unbounded values

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. SLO alerting: burn rates and Alertmanager design

13 minutes. Turn an SLO into multi-window burn rate alerts, route them with Alertmanager so the right people hear about the right problems, and cut alert noise.

Alerts on raw thresholds either miss real trouble or wake people for nothing. Alerting on the speed at which an error budget is being spent fits what users feel and gives you something to decide on.

What you will learn

- Error budgets and burn rate
- Multi-window, multi-burn-rate alerts
- Page versus ticket routing
- Alertmanager grouping, inhibition and silences
- Measuring alert quality

Error budget and burn rate

A 99.9 percent monthly SLO leaves 0.1 percent of requests as budget. Burn rate is the speed of spending it: a burn rate of one uses exactly the whole budget in the month, and a burn rate of 14.4 uses it in about two days. Alerts on high burn rates catch real trouble early.

Multiple windows

A fast alert uses a short window and a high burn rate, such as 14.4 over one hour and five minutes, and pages someone. A slow alert uses a lower burn rate such as 6 over six hours and also 3 over three days, and opens a ticket. Requiring both a long and a short window stops alerts that fire after an incident is over.

Routing

Alertmanager routes by labels to receivers. Page only for symptoms users feel and that need action now. Send everything else to tickets or chat. Include a runbook link and the dashboard in every alert so the responder starts with context.

Grouping, inhibition and silences

Group alerts by service so one outage creates one notification. Use inhibition so a cluster down alert suppresses node level alerts. Use silences with an expiry for planned work, and record who created them and why.

Alert quality

Review each page: was it actionable, urgent and real? Track pages per week per on-call person and the share that needed no action. Delete or fix alerts that fail that review.

Diagram: SLO alerting: burn rates and Alertmanager design

Part	What it does	What goes wrong
SLI query	Error ratio.	Wrong denominator.
Burn rate rules	Fast and slow windows.	Single window.
Alertmanager	Groups and routes.	Bad routing.
On-call	Acts on pages.	Alert fatigue.

Flow: SLI query to Burn rate rules / Burn rate rules to Alertmanager / Alertmanager to On-call

MARGA by EknathaLabs marga.eknathalabs.com

Incident: Two hundred pages from one outage

Incident: *A database failure triggered a separate page for every dependent service.*

```
$ amtool alert query --active | wc -l
214
$ amtool config routes show | head -4
Routing tree:
 receiver: pager
 group_by: [alertname instance]
(this is a simulated illustration, not a real incident)
```

Grouping by instance meant every pod produced its own notification. Grouping by service, adding an inhibition rule from the database alert to dependent alerts and moving most conditions to ticket routes reduced the next incident to three pages.

Common mistakes

- Paging on causes such as CPU instead of user impact
- Using one short window that flaps
- No runbook link in the alert
- Grouping by instance so one outage floods people
- Silences with no expiry or owner

Check yourself

What does a burn rate of one mean?

Answer: The budget will be spent exactly at the end of the period

Why combine a long and a short window?

Answer: To stop alerts firing after the problem has ended

What should a page require?

Answer: A user visible problem that needs action now

What does inhibition do?

Answer: Suppresses lower level alerts while a higher level one is firing

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 4. Logs at scale: Loki, labels and retention

12 minutes. Design a log pipeline that stays affordable, using low cardinality labels, structured fields, sensible retention and queries that filter early.

Logs are the most expensive signal to keep and the one people search under stress. A cost aware design uses a few stable labels, structured content and tiered retention.

What you will learn

- How Loki indexes labels and stores log lines
- Choosing labels: few and stable
- LogQL basics and filtering early
- Retention, tiers and sampling
- Linking logs to traces and metrics

Index labels, not content

Loki indexes only labels such as namespace, app and level, and stores the log text compressed in chunks. This keeps cost low but means every query narrows by labels first and then filters the text. A good label has a small set of values that does not change per request.

Labels to avoid

Do not use user ids, request ids or pod names with churn as labels. Each unique combination creates a new stream, and thousands of tiny streams make ingestion slow and queries costly. Put such values in the log line as structured fields and filter them at query time.

LogQL basics

A query starts with a stream selector such as `{app="api", env="prod"}`, then filters with `|= "error"` or parses with `| json`. Filter lines first and parse later, because parsing every line is expensive. Metrics from logs, such as `rate({app="api"} |= "error" [5m])`, support alerts where no counter exists.

Retention and tiers

Keep recent logs on fast storage and older logs in cheap object storage with a retention period per tenant or stream. Audit and security logs may need a longer period, while debug logs may need a day. Sample or drop noisy health check lines at the agent.

Correlation

Add a trace id to each log line and configure the data source to link it to the trace view. From a spike in a metric you can then jump to logs of that service and onward to a single slow trace.

Diagram: Logs at scale: Loki, labels and retention

Part	What it does	What goes wrong
Log agent	Adds labels, ships lines.	Too many labels.
Distributor	Validates and routes.	Rate limited.
Ingester and index	Builds chunks.	Stream explosion.
Object storage	Chunks and retention.	Cost growth.

Flow: Log agent to Distributor / Distributor to Ingester and index / Ingester and index to Object storage

MARGA by EknathaLabs marga.eknathalabs.com

Incident: Logs rejected for too many streams

Incident: *A new label caused Loki to reject writes and logs went missing during an outage.*

```
$ logcli series '{app="api"}' --analyze-labels | head -4
Label Name  Unique Values  Found In Streams
pod_uid     48211          48211
env         3              48211
(this is a simulated illustration, not a real incident)
```

The agent added the pod uid as a label, so every restart created a new stream and the limit on active streams was hit. Removing that label and keeping app, env and namespace restored ingestion.

Common mistakes

- Using request or user ids as labels
- Keeping all logs at the same retention
- Parsing every line before filtering
- Logging unstructured text that is hard to query
- No link between logs and traces

Check yourself

What does Loki index?

Answer: Labels

Which makes a poor label?

Answer: A request id

How should a query start?

Answer: With a label selector, then line filters

How do you save storage cost?

Answer: Tiered retention and dropping noisy lines at the agent

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. Distributed tracing: sampling and correlation

12 minutes. Use traces to find slow dependencies, choose head and tail sampling sensibly, propagate context across services and link traces to metrics and logs.

A trace follows one request across services. It answers where the time went. Done carelessly, it is expensive and incomplete, so sampling and propagation need deliberate choices.

What you will learn

- Spans, traces and context propagation
- Head sampling versus tail sampling
- Attributes, events and exemplars
- Reading a trace to find the slow hop
- Tracing cost and storage

Spans and propagation

Each unit of work is a span with a start, a duration and attributes. The trace id travels between services in headers, such as the W3C traceparent header. If one service does not forward it, the trace breaks in two. Check message queues and async hops, which often lose context.

Head sampling

The decision is made at the start of the request, for example keep one percent. It is cheap and simple, but it keeps few of the rare, slow or failing requests you most want to see.

Tail sampling

A collector holds the whole trace for a short period and then decides: keep all errors, keep traces slower than a threshold and keep a small share of the rest. It needs memory and all spans of a trace sent to the same collector, but gives far better data per dollar.

Reading a trace

Look for the longest span on the critical path, repeated spans such as the same query run many times and gaps where no span exists. A gap often means queueing or an uninstrumented hop. Compare a slow trace with a normal one.

Exemplars and cost

Exemplars attach a trace id to a metric sample so a latency histogram can link to a real slow trace. Control cost with tail sampling, attribute limits and a retention period of days, not months.

Diagram: Distributed tracing: sampling and correlation

Part	What it does	What goes wrong
Service A	Starts the trace.	Missing header.
Service B	Continues context.	Drops context.
Collector	Tail sampling.	Memory pressure.
Trace backend	Stores and queries.	Cost growth.

Flow: Service A to Service B / Service A to Collector / Service B to Collector / Collector to Trace backend

Incident: Traces that ended at the queue

Incident: *Slow checkouts could not be traced past the message queue.*

```
$ kubectl logs deploy/otel-collector | grep -c "orphan span"
```

MARGA by EknathaLabs
marga.eknathalabs.com

```
1840
$ kubectl logs deploy/checkout | grep traceparent | head -2
no traceparent header found on consumed message
(this is a simulated illustration, not a real incident)
```

The producer did not copy the trace context into the message headers, so the consumer started a new trace. Adding context injection and extraction around the queue joined the traces and revealed the real delay in the payment service.

Common mistakes

- Sampling one percent at random and missing the slow requests
- Forgetting to propagate context through queues
- Putting unbounded values in span attributes
- Keeping traces for months
- Never linking traces to logs and metrics

Check yourself

What carries a trace between services?

Answer: A trace context header such as traceparent

What is the benefit of tail sampling?

Answer: Keeps errors and slow traces

What is an exemplar?

Answer: A trace id attached to a metric sample

What does a gap in a trace often mean?

Answer: Queueing or uninstrumented work

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. Observability as code and cost control

12 minutes. Manage dashboards, alerts and collectors from version control, set ownership and budgets for telemetry, and review cost against value.

Observability grows by accident when each team adds metrics and dashboards by hand. Treating it as code gives review, repeatability and a way to control cost.

What you will learn

- Dashboards and alerts as code
- Collector pipelines in configuration
- Ownership, labels and budgets for telemetry
- Reading the cost by signal
- A review habit that prunes

Dashboards and alerts as code

Store Grafana dashboards as JSON or generated from a library, and alert rules as files, in version control. Deploy them through a pipeline. Reviews catch broken queries and give every change an author. Hand edits in the UI are overwritten at the next deploy, so teams must know that.

Collector pipelines

An OpenTelemetry Collector configuration lists receivers, processors and exporters. Use it to add resource attributes, drop noisy metrics, batch data and route to different backends. Keep the configuration in the same repository as the rules and test it with a staging collector.

Ownership and budgets

Every metric, log stream and dashboard should have an owner label or folder. Give each team a budget in series, log gigabytes a day or trace spans a day, and show their usage next to it. Limits at the collector stop a single mistake from becoming a large bill.

Reading cost by signal

Cost comes from active series for metrics, ingested gigabytes for logs and sampled spans for traces. Find the top ten contributors in each. Often a few metrics with unbounded labels or a chatty debug log make most of the cost.

Pruning review

Each quarter, list dashboards with no views and alerts that never fired or always fire, and decide to delete or fix them. Track the telemetry cost per request or per customer so growth is visible against traffic.

Diagram: Observability as code and cost control

Part	What it does	What goes wrong
Repository	Rules and dashboards.	Hand edits.
Pipeline	Validates and deploys.	No tests.
Collector	Limits and routes.	Bad config.
Cost report	Usage by owner.	No owner.

Flow: Repository to Pipeline / Pipeline to Collector / Collector to Cost report

Incident: A dashboard edit that disappeared

Incident: *A team fixed a panel in the UI and the fix vanished the next day.*

MARGA by EknathaLabs
marga.eknathalabs.com

```
$ git log --oneline -3 -- dashboards/payments.json
a1b2c3d Add latency panel
$ curl -s -u admin:$GRAFANA_PASS localhost:3000/api/dashboards/uid/payments | grep -c version
1
(this is a simulated illustration, not a real incident)
```

The dashboard is provisioned from the repository, so the UI edit was overwritten by the next deploy. The fix was committed as a pull request, and the provisioning settings now mark UI editing as disabled with a note linking to the repository.

Common mistakes

- Editing provisioned dashboards by hand
- No owner on metrics and dashboards
- Never reviewing unused alerts
- Keeping debug logs at full volume in production
- Treating telemetry cost as someone else problem

Check yourself

Why keep dashboards in version control?

Answer: Review, history and repeatable deploys

Where can noisy metrics be dropped centrally?

Answer: In the collector pipeline

Which usually drives most metric cost?

Answer: A few metrics with unbounded labels

What should a quarterly review do?

Answer: Delete or fix unused and noisy alerts and dashboards

MARGA by EknathaLabs
marga.eknathalabs.com

Cheat sheet: Observability in depth

PromQL patterns, burn rate pairs and rule checks.

PromQL patterns

Goal	Query
Request rate	`sum(rate(http_requests_total[5m])) by (service)`
Error ratio	`sum(rate(http_requests_total{code=~"5.."}[5m])) / sum(rate(http_requests_total[5m]))`
p99 latency	`histogram_quantile(0.99, sum(rate(http_request_duration_seconds_bucket[5m])) by (le))`
Series count	`count({__name__=~".+"})`

Burn rate pairs

Severity	Long window	Short window	Rate
Page	1 hour	5 minutes	14.4
Page	6 hours	30 minutes	6
Ticket	3 days	6 hours	1

Checks

Check	Command
Validate rules	`promtool check rules rules.yml`
Test rules	`promtool test rules tests.yml`
Label values	`logcli labels`

MARGA by EknathaLabs marga.eknathalabs.com